

void function(array, $\downarrow$ $\int_{int} \times \max$, $\downarrow$ $\int_{int} \times \min$, int size)

calculations:

max = }
min = }

main() { arr = [1, 2, 5, -1, 4, 3, 0, 22, 9, 7]

address → max = arr[0]
address → min = arr[0]
function(arr, &max, &min, size);

int $\times$ min &
Home
int $\times$ Friend

CBR

Structures & Unions

User-Defined Data Types

(Anvitha)

name v1 → M1 A ✓
age v2 → M2 23 ✓
rollNo v3 → M3 121 ✓

Struct

Size = Total
= sizeof(v1+v2+v3)

Memory Allocation

(Chaitanya)

name v1 → M1
age v2 → M2
rollNo v3 → M3

Union

Size = sizeof(largest)

5! = 5 × 4 × 3 × 2 × 1 = 120

1! = 1

0! = 1

[-1! = Invalid]

fact(int n)

if n == 1 || n == 0
return 1;

if n < 0
return -1;

else

Recursion: → n → n-1 ⇒ n = 5

factorial of 0 → 1

factorial of 1 → 1

factorial of (-ve) → -1 (invalid)

Recursion Tree:

return n × fact(n-1)

fact(5)

5 × fact(4)

5 × fact(3)

5 × 4 × fact(2)

5 × 4 × 3 × fact(1)

5 × 4 × 3 × 2 × 1 = 120

Back Tracking

T, U, H

DFS

n < 0
return -1

n == 0 || n == 1
return 1

n > 1
return n × f(n-1)

* Fibonacci Series: → Normal → Execute → Code

0, 1, 1, 2, 3, 5, 8, 13, 21

n1 = 0
n2 = 1

n1 = n2 n2 = n3

n3 = n1 + n2

code → What's app

Recursion → Tree Diagram

Transpose ✓
UTM ✓
LTM ✓
B to D

n = 15 = 0

n % 2 = 15 % 2 = 1

int n = n / 2 = 7

7 % 2 = 1

7 / 2 = 3

3 % 2 = 1

3 / 2 = 1

1 % 2 = 1

1 / 2 = 0

UTM
LTM
Transpose
C++
Fibonacci

arr[10]

for(i = 0; i < n; i++)

arr[i] = n % 2;

n = n / 2;

for(i = i - 1; i >= 0; i--)

a[i]

arr

0 1 2 3 4

1 1 1 1